

What is the difference between whisper-ctranslate2 и whisper.cc

• Both are reimplementations of OpenAI's Whisper model, optimized for inference speed, but with very different approaches: whisper.cpp

- Language: Pure C++ (no Python required)
- Engine: Uses the GGML tensor library (same author, Georgi Gerganov)
- Models: GGML format (converted from OpenAI originals)
- GPU: Metal (Apple), CUDA, Vulkan, OpenCL
- Strengths: Minimal dependencies, single binary, excellent on CPU and Apple Silicon, easy to embed in native apps
- Use case: Desktop apps, embedded systems, CLI tools — exactly what's in your project whisper-ctranslate2
- Language: Python CLI wrapping the faster-whisper library
- Engine: CTranslate2 — a C++ transformer inference engine with Python bindings
- Models: CTranslate2 format (int8/float16 quantized)
- GPU: CUDA (primary focus)
- Strengths: Very fast on NVIDIA GPUs, word-level timestamps, built-in VAD (silero-vad), drop-in replacement for OpenAI's whisper CLI
- Use case: Server-side batch transcription, GPU workloads, Python pipelines Summary

	whisper.cpp	whisper-ctranslate2
Runtime	C++, no Python	Python + CTranslate2
Best hardware	CPU, Apple Silicon	NVIDIA GPU
Dependencies	Minimal	Python ecosystem
VAD built-in	No (external)	Yes (silero)
Embeddable	Yes	Not easily

For your Electron app with live transcription, whisper.cpp is the right call — no Python dependency, ships as a binary, and works well on CPU/Metal. whisper-ctranslate2 would be more relevant if you were building a Python server processing audio in bulk on a GPU machine.

Are they use same binary models?

No, they use different model formats: whisper.cpp

- Uses GGML format (.bin files)
- Models must be converted from OpenAI's original weights using the convert-pt-to-ggml.py script, or downloaded pre-converted from Hugging Face (ggerganov's repo)
- Example: ggml-base.en.bin whisper-ctranslate2 / faster-whisper
- Uses CTranslate2 format (a directory with model.bin, config.json, tokenizer.json, etc.)
- Models are converted using ct2-opus-converter or downloaded pre-converted from Hugging Face (guillaumekln's repo)
- Typically quantized to int8 or float16 Common origin Both ultimately derive from the same OpenAI PyTorch checkpoints (.pt files), but the conversion pipelines are separate and the resulting files are

not interchangeable. So if you have a model for whisper.cpp, you can't use it with faster-whisper and vice versa. You'd need to convert from the original OpenAI weights or download the right pre-converted version for each. > Well, to transcribe audio file to text my application uses whisper-ctranslate2, for life transcribe whisper.cc. I think that it's need uniform solution. What do you think? • Let me look at how both are currently used in the code before giving a recommendation. Searched for 2 patterns • Good picture. My recommendation: consolidate on whisper.cpp. Reasons:

- 1. Already embedded — whisper.cpp binary is already in the project for live transcription, no extra install needed
- 2. No Python dependency — whisper-ctranslate2 requires pip install, which is fragile for a desktop app (user must have Python + the package installed)
- 3. whisper.cpp transcribes files too — the main whisper binary (not whisper-stream) takes an audio file and outputs text, same as whisper-ctranslate2
- 4. Same model — you're already downloading GGML models for live transcription, file transcription reuses them for free
- 5. Consistent config — one model setting covers both modes The only downside: whisper-ctranslate2 has slightly better speed on NVIDIA GPU via CUDA. If your users are mostly on L

Whisper.cpp GGML Model Comparison

Sizes (quality vs speed)

Model	Params	Notes
tiny	39M	Very fast, poor accuracy
base	74M	Fast, acceptable for simple audio
small	244M	Good balance
medium	769M	Good quality
large-v2	1.5B	Best quality (older)
large-v3	1.5B	Best quality (current)
large-v3-turbo	~800M	Distilled from large-v3, nearly same quality, 2x faster

Quantization suffixes (size vs quality within same model)

Suffix	Bits	RAM vs F16	Quality loss
(none)	F16	100%	none
q8_0	8-bit	~55%	barely noticeable
q5_0	5-bit	~35%	small
q5_1	5-bit	~35%	small
q4_0	4-bit	~25%	noticeable

Recommendations for Russian, desktop app

- **Never use .en models** — English-only, Russian won't work
- For **file transcription**: `ggml-large-v3-turbo-q5_0` — best quality/speed ratio, ~1GB RAM
- For **live transcription**: `ggml-small-q5_0` or `ggml-medium-q5_0` — low latency required, large models add too much delay
- **Sweet spot**: `q5_0` quantization — negligible quality loss vs F16, much smaller and faster
- Avoid `q4_0` for Russian — quantization artifacts more noticeable on non-English speech